AgentShield + SecureAI

Complete technical dossier: what we built, what it fixes, how it is wired, the full stack, and every key explained

Fact	Value
Repository	github.com/namankrag/cyber_hack
Branch in this dossier	final-merge (local head 8125b4a), pushed to origin/final-merge
Integration base	muse-secureai-integration @ 0d2e448 (pushed); frontend-muse @ 8034eae merged in
Test suite	268 passed, 1 skipped (semgrep engine test; graceful AST fallback)
Evaluation	secureai eval --once: 8/8 gates at threshold 0.5 (tuned: 0.35-0.45 fail one gate)
Web console	Next.js production build passes; lint + typecheck clean; admin-gated proxy verified live
Date / scope	31 Oct 2026. Synthetic data only. No real secrets in code or docs.

Contents: 1 What we are working on 2 Problems solved, with evidence 3 Architecture and connections 4 Tech stack 5 Every API key and setting, in detail 6 Verification snapshot 7 Remaining work

1 What we are working on

PS-01 **AgentShield** is a defensive layer that sits between an AI agent and its tools. Every tool call passes a policy enforcement point **before** the tool runs. The verdict is one of **allow / deny / require_approval** (human confirmation for high-impact calls), with inspectable ordered reasons and an append-only audit trail. Decisions are **fail-closed**: any error, timeout, unknown tool, or missing approval resolves to deny. It distinguishes legit tasks from injected instructions in untrusted content, blocks unsafe tool actions, and holds sensitive calls for a human.

PS-02 **PatchProof + SecureAI** is the code-security layer on the same repo. A deliberately vulnerable practice app (**VulnBank**, plus a second **ShopAPI** fixture) carries planted CWEs. PatchProof scans (AST + Semgrep rules), **proves** each finding with a working exploit in an isolated sandbox, applies a minimal patch to a copy, and only accepts it when the exploit is blocked **and** all regression checks pass. A **closed remediation loop** repeats scan→exploit→patch→verify until zero confirmed vulnerabilities (observed 5→0 in 5 iterations). An **independent grader** rechecks everything into a JSON/HTML **scorecard** with enforced gates (nonzero exit on failure).

The **web console** (Next.js) is the judges' window onto both layers: live decision feed, enforcement switch, approval center with Approve/Deny, benchmark panel, and a red-team lab that runs an ON-vs-OFF enforcement comparison in an isolated harness without touching the global switch.

Current state: the **final-merge** branch unites the hardened AgentShield runtime, the SecureAI/PatchProof platform, and the web console. The merge kept the audited backend **app.py** (rate limit, body caps, security headers, constant-time key compare, docs gate) and added the console routes on top.

2 Problems solved, with evidence

Problem	Fix (evidence)
Prompt injection via retrieved docs	Heuristic detector (normalized, multilingual HI/KN/TA/TE) + untrusted-source rule; 90/500 denied in live audit; ATTACK-01/02 denied in eval.
Unsafe tool actions	Most-restrictive-wins 8-step engine; transfer_funds allow:false; permission checks per tool/source.
No human in the loop	Single-use hash-bound approvals; queue + CLI + console Approve/Deny; 43 pending held correctly.
Silent failures	Fail-closed everywhere incl. generators (TypeError), timeouts, unknown tools; fail_closed recorded per verdict.
Uninspectable verdicts	Ordered reasons + matched detector quotes in every response, dashboard, and audit JSONL.
Audit tampering/loss	Append-only JSONL, secret redaction, sinks interface (JSONL default; Postgres/Redis behind flags); build-info manifest.
Scanner noise (PS-02)	Exploit-confirmation gate: 16 raw alerts → 8 confirmed → 8 fixed; decoys dropped as NOT CONFIRMED.
Unproven patches	85/85 exploit variants blocked incl. encoded/double-encoded/XSS/IDOR; weak blocklists rejected as PATCH BYPASSABLE.
Over-blocking regressions	Extended suites (apostrophes, unicode, legit hostnames, nested public files) + per-function coverage in report.
Vague severity	CVSS 3.1 vectors + base scores with per-metric justification; diff size shown, oversized patches flagged.
Admin takeover via console (F1/F2)	Server-side admin session (httpOnly cookie, ADMIN_PASSWORD); Origin + JSON checks on non-GET; verified 401→200.
Proxy smuggling (F3/F4)	Exact method+route allowlist; ./.. and encoded-slash rejection; 64KB cap with 413.
Browser-chosen trust (F5)	Trust label forced from server-side scenario; browser value ignored.
Lockfile ships Windows-only pin (B1)	pywin32 marked sys_platform=='win32'; hash-enforced installs verified in a fresh venv.
Stale fixture hashes (B2)	Fixtures manifest + refresh command; suite back to 268+1 green.
Audit of audit (B6)	Evidence reads build-time build-info JSON; no runtime git; counts always match the tree.

3 Architecture and connections

One request, one boundary. Every path below funnels through the same policy engine; the differences are only transport (in-process call, HTTP, MCP) and surface (agent runtime vs practice-app code vs browser).

3.1 Runtime check (the hot path)

```
AI agent  --ToolCall(tool, args, source, context)-->  ENFORCEMENT POINT
     |                                                          |
     |   1 normalize -> 2 detectors (heuristic + optional LLM)  |
     |   3 policy (sensitivity x source x threat, most-restrictive-wins)
     |   4 provenance resolve (token/nonce/expiry vs harness trust)     |
     +-- DENY / REQUIRE_APPROVAL (hold) / ALLOW <-- audit sink (JSONL)  |
     |                                                          |
     +-- approval? single-use hash-bound approve/deny (queue/CLI/UI) ---+
Transports: guard() in-process | POST /v1/check | MCP middleware (tools/call).
```

3.2 Retrieval (RAG) path

```
agent query --> read_rag_docs (low-sensitivity tool, still policed)
    --> Chroma PersistentClient, 14-doc UNTRUSTED corpus, retrieval scoped untrusted-only
    --> retrieved text re-enters the SAME boundary as context (poisoned doc => deny + quoted rule)
Agentic graph: retriever -> gatekeeper -> reporter (LangGraph StateGraph, interrupt() on approval).
```

3.3 Console path (browser)

```
browser --> Next.js proxy /api/[...path] (exact allowlist, 64KB cap, CSRF Origin+JSON checks)
    --> admin routes attach ADMIN_KEY only with httpOnly session cookie (ADMIN_PASSWORD login)
    --> FastAPI backend (same engine) --> panels: feed, switch, approvals, benchmark, red-team lab.
```

3.4 Code-security loop (PatchProof / SecureAI)

```
vulnbank/app.py + shopapi/app.py (synthetic fixtures, fixtures.json sha256 manifest)
    --> scanner (AST + Semgrep) : 16 alerts --> exploit-confirm (sandboxed, 10s deadline)
    --> 8 confirmed --> curated patch on isolated copy --> 85/85 variants blocked + regressions green
    --> 8 fixed & verified --> CVSS + coverage + SARIF --> loop repeats to 0 residual (5 iters)
Bridge: runtime DENY on a governed tool triggers a source scan of the registered file (links=tool name).
Grader recomputes all gates from raw evidence; scorecard.json/html; eval --tune picks threshold 0.5.
```

4 Tech stack

Layer	Technology (pinned)	Role
Runtime	Python 3.11	Single interpreter for gateway, engine, loop, fixtures.
API	FastAPI + uvicorn[standard]	Gateway: /v1/check, audit, evidence, approvals, admin, redteam, metrics.
Data validation	pydantic v2	ToolCall, Decision, audit records, API models.
Policy	YAML policies + SHA256 trust pin	policies/default.yaml; default_policy.yaml; digest fail-closed.
Retrieval	ChromaDB PersistentClient (local ONNX)	14-doc UNTRUSTED corpus; no hosted DB, no key.
Agent graph	LangGraph StateGraph + SqliteSaver	retriever/gatekeeper/reporter; interrupt() HITL approvals.
Detection	Heuristic engine (own code) + optional Gemini/Groq LLM	Normalized multilingual patterns; LLM keyed, keyless default.
Fixture apps	Flask (VulnBank/ShopAPI) + Flask test client	Planted CWEs; exploits/regressions run in-process sandbox.
Console	Next.js 15 + React 19 + Tailwind	Workspace UI; admin-gated server-side proxy; prod build verified.
Tests/quality	pytest (268+1), Semgrep rules, SARIF, GitHub Action	Gates: suite green, 8/8 eval, lint + tsc + build.
Reproducibility	pip-tools lockfile + --require-hashes; Docker + render.yaml	270KB+ hashed lock; content-hashed fixture manifest.

5 Every API key and setting, in detail

Principle: the enforcement core runs with **no keys at all**. Keys only (a) establish trust roots, (b) gate access, or (c) enable the optional semantic layer. Anything marked optional degrades to a documented fail-closed or flagged behavior when unset.

5.1 Trust root (who may assert tool-call provenance)

Variable	Used by / effect	If unset
AGENTSHIELD_HARNESS_SECRET	HMAC for provenance tokens (v1: nonce+expiry+session) and the signed tool manifest; verification on every check and tool registration.	INSECURE DEV MODE: self-asserted source honored, flagged in /v1/evidence and audit. Never for production.

5.2 Gateway access control

Variable	Used by / effect	If unset
AGENTSHIELD_API_KEY	X-API-Key required on every HTTP endpoint incl. audit/evidence when set.	Open dev gateway (auth_required=false); everything reachable, flagged.
AGENTSHIELD_ADMIN_KEY	/v1/admin/* (enforce switch, reset) requires THIS key instead of the API key.	Admin routes fall back to the API key; unset both = open dev mode.
AGENTSHIELD_ENFORCE	Master ON/OFF for verdicts (OFF still records would-be verdicts for the benchmark).	Defaults ON.

5.3 Detectors and semantic layer

Variable	Used by / effect	If unset
AGENTSHIELD_DETECTORS	Comma list, e.g. heuristic or heuristic,llm (default heuristic).	Heuristic only; keyless and offline.
AGENTSHIELD_LLM_PROVIDER	Which LLM detector: gemini or groq (used only if detectors includes llm).	No LLM path.
GEMINI_API_KEY	Google Gemini free-tier key for the semantic detector.	LLM detector unavailable; heuristic still blocks.
GROQ_API_KEY	Groq free-tier fallback key.	Same as above.
AGENTSHIELD_INJECTION_THRESHOLD	Score 0-1 at/above which a detection counts as a threat (tuned: 0.5).	0.5. Lower over-blocks approvals; higher misses paraphrase.
AGENTSHIELD_LLM_TIMEOUT_S	Timeout for the LLM call; expiry fails closed to heuristic-only.	8.0s.

5.4 Policy, audit, limits

Variable	Used by / effect	If unset
AGENTSHIELD_POLICY_PATH	Active YAML policy (default policies/default.yaml).	Bundled default.
AGENTSHIELD_POLICY_SHA256	Pinned digest of the policy file; mismatch fails closed (trust root).	No pin check.
AGENTSHIELD_AUDIT_PATH	Append-only JSONL trail (default audit.log.jsonl).	Local file; attach a Render disk in prod.
AGENTSHIELD_REDTEAM_AUDIT_PATH	Isolated red-team audit file (default audit.redteam.jsonl).	Same directory as audit trail.
AGENTSHIELD_REDACT_KEYS	Extra argument keys to redact (defaults cover password/secret/token/api_key/...).	Defaults apply.
AGENTSHIELD_RATE_LIMIT_PER_MIN	Fixed-window per-IP backstop (600); /health+/ready exempt; 429+Retry-After.	600. Edge limiter remains the real boundary.
AGENTSHIELD_ENABLE_DOCS	1 keeps /docs+/redoc+/openapi.json (judge demos); 0 closes them in prod.	1.
AGENTSHIELD_RAG_PATH	Chroma persistent directory for the corpus.	Ephemeral dir (re-ingested per run).

Variable	Used by / effect	If unset
AGENTSHIELD_BENCH_DIR	Where benchmark result JSONs land (default bench/).	bench/.
AGENTSHIELD_APPROVAL_BACKEND	memory (default) | postgres | redis for the approval store.	In-memory; multi-process needs redis/postgres.
AGENTSHIELD_REDIS_URL	Redis URL (fail-closed if missing/unreachable when selected).	N/A unless selected.
AGENTSHIELD_POSTGRES_DSN	Postgres DSN for JSONB approval payloads (fail-closed if missing).	N/A unless selected.
AGENTSHIELD_UPSTREAM_CMD	Stdio command of the upstream MCP server for drop-in proxy mode.	Built-in 5 demo tools; no customer change needed otherwise.

5.5 Console (Next.js, server env only — never NEXT_PUBLIC_*)

Variable	Used by / effect	If unset
AGENTSHIELD_API_URL	Backend origin the proxy forwards to (verified http://127.0.0.1:8000).	Proxy default; set explicitly per deploy.
AGENTSHIELD_API_KEY (server)	Key the proxy attaches to backend calls.	Backend dev-open required.
AGENTSHIELD_ADMIN_KEY (server)	Key attached to admin routes, only with a valid admin session.	Admin calls 401.
ADMIN_PASSWORD	Password for /api/admin/login → httpOnly signed session cookie.	No login possible; admin UI stays locked (safe default).
PUBLIC_ORIGIN	Accepted Origin for cross-origin non-GET (CSRF check).	Same-origin only.

5.6 Demo-only secrets (synthetic, never real)

Variable	Used by / effect	If unset
SHOPAPI_ADMIN_KEY	Backing secret inside the synthetic ShopAPI fixture (deserialization/secret demos).	Fixture still runs; one demo path degrades.
Demo login agentshield-demo	Local console password used in this dossier's live run (env-only, not committed).	Any ADMIN_PASSWORD works; set your own.

6 Verification snapshot (observed, final-merge)

Gate	Result
pytest -q (root)	268 passed, 1 skipped (Semgrep test; AST fallback).
secureai eval --once	8/8 gates at threshold 0.5; tune: 0.35/0.40/0.45 fail exactly one gate each.
Closed loop	CONVERGED 5→0 residual, 5 iterations; deployed-copy re-attack clean.
Platform demo	1 runtime attack blocked, 5 code vulns fixed & verified, 1 auto scan.
Benchmark (84 frozen cases)	OFF 100% attack success → ON 4.3%; FP 0.0%; false-hold 0.0%; p50 0.58ms p99 2.77ms.
Load (async, one box)	~620 req/s served (p99 ~56-108ms); 4,028 req/s in-process.
Web	npm ci ok; lint clean; tsc clean; production build success; proxy live-probed.
Live probes	Backend :8000 all 200; console :3000 all 200; admin 401→200 with session; red-team run 200 with full ON/OFF comparison.

Run it: **python -m pytest -q** (root) · **python -m secureai eval --once --out-dir reports** · backend **uvicorn agentshield.gateway.app:app --port 8000** · console **npm start -- -p 3000** in web/ (AGENTSHIELD_API_URL=http://127.0.0.1:8000, ADMIN_PASSWORD of your choice).

7 Remaining work (honest list)

1) Docker image build is unverified on this box (no Docker; CI runs it). 2) GitHub Action + Render deploy are configured but their first green runs happen in CI/on Render, not here. 3) Historical doc snapshots intentionally keep old counts as dated records. 4) Chroma upstream CVEs have no fix; containment (embedded-only use, no server/API paths) is documented. 5) Multi-process deploys need the Redis/Postgres backends + sticky sessions (interfaces shipped, wiring is deploy-time).

Generated from the final-merge tree. Every number above was observed in this session; nothing is projected. Synthetic data only; no real secrets appear in code, docs, or this PDF.